

Teste Estrutural

Do Código ao Grafo de Fluxo de Controle

McCabe e Teste de Caminho Básico

Aula introdutória com exemplos, soluções e exercícios

Objetivos da aula

- Observar um código e identificar fluxo de controle
- Transformar código em Grafo de Fluxo de Controle
- Identificar nós, arestas e decisões
- Calcular a Complexidade Ciclomática de McCabe
- Identificar caminhos linearmente independentes
- Derivar casos de teste para percorrer caminhos

De onde estamos vindo?

No teste funcional, pensamos de fora para dentro.

Entrada -> Software -> Saída

A pergunta principal era: para determinada entrada, qual deveria ser a saída?

Agora mudamos a pergunta

Em vez de perguntar somente se a saída está correta, passamos a perguntar:

O que aconteceu dentro do programa para produzir essa saída?

Teste estrutural

No teste estrutural conhecemos a implementação.

Observamos:

- instruções
- decisões
- desvios
- repetições
- caminhos de execução

Por que representar o código como um grafo?

O código-fonte mostra o que foi escrito.

O Grafo de Fluxo de Controle (CFG) mostra como a execução pode percorrer o que foi escrito.

Essa abstração é necessária porque a numeração das linhas não revela, sozinha, os diferentes comportamentos possíveis.

CÓDIGO-FONTE



identificar blocos e decisões



GRAFO DE FLUXO DE CONTROLE



visualizar caminhos possíveis



definir critérios de cobertura



derivar casos de teste

O grafo transforma a lógica do programa em uma estrutura que podemos observar, contar e testar.

O que o CFG torna explícito?

Com o grafo conseguimos visualizar:

- quais instruções sempre são executadas;
- onde o fluxo pode se dividir;
- onde caminhos diferentes voltam a se encontrar;
- quais estruturas podem repetir a execução;
- quais trechos podem não ser alcançados;
- quais caminhos precisam de entradas específicas para serem percorridos.

Portanto, o CFG cria uma ponte entre:

estrutura do código → caminhos de execução → casos de teste

Por que isso é importante para teste estrutural?

Sem o CFG, podemos escolher testes apenas por intuição.

Com o CFG, conseguimos justificar estruturalmente os testes:

"Este caso existe porque precisamos percorrer esta aresta."

"Este outro caso existe porque precisamos exercitar o resultado falso desta decisão."

"Precisamos de outro caminho porque ele introduz uma aresta ainda não exercitada."

O teste deixa de ser apenas uma coleção de entradas e passa a ser relacionado explicitamente à estrutura interna do programa.

Grafo não é o programa: é uma abstração

Não precisamos representar cada detalhe sintático.

Agrupamos instruções sequenciais em blocos básicos e destacamos os pontos que alteram o fluxo.

Em geral, interessam especialmente:

- decisões (if, else if, switch);
- condições de repetição (for, while, do while);
- retornos;
- chamadas que alteram a continuidade do fluxo;
- exceções, quando fizerem parte do escopo da análise.

O objetivo não é desenhar o código linha por linha.

O objetivo é representar as alternativas de execução.

A pergunta que o grafo responde

Ao olhar para um CFG, queremos conseguir responder:

1. Onde o fluxo começa?
2. Quais caminhos podem ser seguidos?
3. Onde existem decisões?
4. Quais caminhos voltam a se encontrar?
5. Onde existem ciclos?
6. Como o fluxo termina?
7. Que entrada faz o programa percorrer cada caminho?

Essa última pergunta conecta diretamente o grafo ao caso de teste.

O amadurecimento esperado

O aluno precisa parar de olhar o código como uma lista de linhas.

Ele deve enxergar o código como uma rede de caminhos possíveis.

Linha de código -> bloco -> decisão -> caminho -> caso de teste

Código não é apenas sequência

Quando programamos, lemos linha por linha.

Quando testamos estruturalmente, perguntamos:

- existe desvio?
- existe retorno antecipado?
- existe loop?
- existe caminho que nunca executa?
- qual entrada percorre cada caminho?

Grafo de Fluxo de Controle

O comportamento do código pode ser representado por um Grafo de Fluxo de Controle.

Nos representam instruções ou blocos.

Arestas representam transferencias possíveis de controle.

Processo central da aula

Sempre faremos a transformacao abaixo.

flowchart LR

A[Codigo] --> B[Grafo de Fluxo de Controle]

B --> C[McCabe]

C --> D[Caminhos independentes]

D --> E[Entradas]

E --> F[Casos de teste]

Representacao 1 - Sequência

Uma sequência simples não cria ramificacao.

Tudo que entra no primeiro bloco obrigatoriamente segue para o proximo.

```
int calcular(int x) {  
    int r = x * 2;  
    r++;  
    return r;  
}
```

flowchart TD

```
A([Inicio]) --> B[Instrucao 1]  
B --> C[Instrucao 2]  
C --> D[Instrucao 3]  
D --> E([Fim])
```

Leitura estrutural da sequência

- Não ha decisão.
- Existe apenas um caminho principal.
- Um teste ja percorre todas as instruções sequenciais.

McCabe: $V(G) = 1$

Representacao 2 - Condicional simples

Um if sem else cria dois caminhos: executar o bloco ou pular o bloco.

```
if (idade >= 18) {  
    autorizado = true;  
}  
return autorizado;
```

```
flowchart TD  
    A([Inicio]) --> B{condicao?}  
    B -- true --> C[Bloco executado]  
    B -- false --> D[Proxima instrucao]  
    C --> D  
    D --> E([Fim])
```

Leitura estrutural do if

A condição gera duas arestas:

- true: entra no bloco
- false: pula o bloco

McCabe: 1 decisão + 1 = 2

Representacao 3 - if / else

O if/else também possui duas saídas, mas ambas possuem bloco explicito.

```
if (a > b) {  
    maior = a;  
} else {  
    maior = b;  
}  
return maior;
```

flowchart TD

```
A([Inicio]) --> B{condicao?}  
B -- true --> C[Bloco if]  
B -- false --> D[Bloco else]  
C --> E[Proxima instrucao]  
D --> E  
E --> F([Fim])
```

Leitura estrutural do if/else

Do ponto de vista de fluxo:

- um teste deve fazer a condição verdadeira;
- outro teste deve fazer a condição falsa.

Apos os blocos, o fluxo normalmente se encontra novamente.

Representacao 4 - while

No while, a condição é avaliada antes do corpo.

Por isso, o corpo pode executar zero vezes.

```
while (i < n) {  
    soma += i;  
    i++;  
}  
return soma;
```

flowchart TD

```
A([Inicio]) --> B{condicao while?}  
B -- true --> C[Corpo do while]  
C --> B  
B -- false --> D[Proxima instrucao]  
D --> E([Fim])
```

Leitura estrutural do while

- Se a condição for falsa no início, o corpo não executa.
- Se for verdadeira, entra no corpo.
- Ao final do corpo, volta para a condição.

Casos importantes: 0 iterações, 1 iteração, mais de 1 iteração.

Representacao 5 - for

O for pode ser visto como uma combinacao de inicializacao, condição, corpo e incremento.

```
for (int i = 0; i < n; i++) {  
    total += i;  
}  
return total;
```

flowchart TD

```
A([Inicio]) --> B[inicializacao]  
B --> C{condicao do for?}  
C -- true --> D[Corpo do for]  
D --> E[incremento]  
E --> C  
C -- false --> F[Proxima instrucao]  
F --> G([Fim])
```

Leitura estrutural do for

Estruturalmente, o for possui uma decisão principal: a condição de permanência no laço.

O incremento cria a aresta de retorno para a condição.

Representacao 6 - do/while

No do/while, o corpo executa antes da primeira avaliacao da condição.

Logo, o corpo executa pelo menos uma vez.

```
do {  
    total += i;  
    i++;  
} while (i < n);  
return total;
```

flowchart TD

```
A([Inicio]) --> B[Corpo do do-while]  
B --> C{condicao?}  
C -- true --> B  
C -- false --> D[Proxima instrucao]  
D --> E([Fim])
```

Leitura estrutural do do/while

- Não existe caso com zero execuções do corpo.
- Existe caminho com uma execução.
- Existe caminho com mais de uma execução.

Isso diferencia o do/while e o for.

Representacao 7 - switch/case

O switch cria multiplas alternativas a partir de uma expressao.

```
switch (tipo) {  
    case 1: r = "A"; break;  
    case 2: r = "B"; break;  
    default: r = "Outro";  
}  
return r;
```

flowchart TD

```
A([Inicio]) --> B{switch(expressao)}  
B -- case A --> C[Bloco case A]  
B -- case B --> D[Bloco case B]  
B -- default --> E[Bloco default]  
C --> F[Saida do switch]  
D --> F  
E --> F  
F --> G([Fim])
```

Leitura estrutural do switch

Cada case representa uma saída possível da decisão.

Para cobertura estrutural, normalmente queremos um caso de teste por alternativa relevante.

Cuidado: sem break, pode ocorrer queda de fluxo entre cases.

Representacao 8 - função/método

Uma função ou método pode ser visto como um subgrafo chamado por outro ponto do programa.

```
double calcularTotal(double valor) {  
    if (valor > 100) return valor * 0.9;  
    return valor;  
}
```

```
flowchart TD  
    A([Chamador]) --> B[Funcao/metodo]  
    B --> C[Processamento interno]  
    C --> D{decisao interna?}  
    D -- true --> E[retorno 1]  
    D -- false --> F[retorno 2]  
    E --> G([volta ao chamador])  
    F --> G
```

Leitura estrutural de função/método

- O método tem entrada.
- Pode possuir decisões internas.
- Pode possuir múltiplos retornos.
- Cada retorno representa uma saída possível do subgrafo.

Em teste de unidade, geralmente testamos o método como unidade isolada.

Representacao 9 - classe

Uma classe não é um único caminho de execução.

Ela organiza estado e comportamento.

```
class Carrinho {  
    private double total;  
  
    void adicionar(double valor) { ... }  
    double fecharCompra() { ... }  
}
```

flowchart TD

```
A[Objeto/Classe] --> B[Método 1]  
A --> C[Método 2]  
B --> D[Estado/atributos]  
C --> D  
D --> E[Retorno/efeito observável]
```

Leitura estrutural de classe

Para teste estrutural, normalmente analisamos:

- cada método individualmente;
- interações entre métodos quando o estado da classe influencia o comportamento;
- caminhos causados por atributos internos.

Classe -> métodos -> grafos de fluxo.

OO real - o que muda?

Em código orientado a objetos, normalmente não existe apenas um grafo.

Temos camadas de análise:

- grafo de relacionamento entre classes;
- grafo de chamadas entre métodos;
- grafo de fluxo de controle dentro de cada método;
- possível análise integrada quando um método chama outros métodos.

flowchart LR

A[Grafo de relacionamento] --> B[Grafo de chamadas]

B --> C[CFG dos metodos]

C --> D[McCabe por metodo]

D --> E[Casos de teste]

E --> F[Cobertura integrada]

OO real - exemplo de dominio

Sistema simples de fechamento de pedido.

Classes envolvidas:

- PedidoService coordena o caso de uso;
- Pedido possui itens e UF de entrega;
- ItemPedido calcula seu total;
- Cliente informa se e VIP;
- CalculadoraDesconto aplica regras de desconto;
- CalculadoraFrete aplica regra por UF.

flowchart TD

```
PS[PedidoService] --> CD[CalculadoraDesconto]
```

```
PS --> CF[CalculadoraFrete]
```

```
PS --> P[Pedido]
```

```
PS --> C[Cliente]
```

```
P --> I[ItemPedido]
```

```
P --> UF[UF entrega]
```

```
C --> VIP[atributo vip]
```

Código OO - classes principais

Observe qué o comportamento esta distribuido entre objetos.

```
class Cliente {
    boolean vip;
}

class ItemPedido {
    double valor;
    int quantidade;
    double total() { return valor * quantidade; }
}

class Pedido {
    List<ItemPedido> itens;
    String uf;
}
```

Código OO - regras distribuidas

As decisões relevantes não estão todas no mesmo método.

```
class CalculadoraDesconto {
    double calcular(Cliente c, double subtotal) {
        if (c.isVip()) return subtotal * 0.10;
        if (subtotal >= 500) return subtotal * 0.05;
        return 0;
    }
}

class CalculadoraFrete {
    double calcular(Pedido p) {
        switch (p.getUf()) {
            case "PR": return 20;
            case "SP": return 35;
            default: return 50;
        }
    }
}
```

Código OO - método de serviço

Método que coordena os objetos. Este é o ponto de entrada do caso de uso.

```
class PedidoService {
    private final CalculadoraDesconto descontos;
    private final CalculadoraFrete fretes;

    double fecharPedido(Pedido p, Cliente c) {
        if (p.getItems().isEmpty()) {
            throw new IllegalArgumentException("pedido vazio");
        }

        double subtotal = p.subtotal();
        double desconto = descontos.calcular(c, subtotal);
        double frete = fretes.calcular(p);
        double total = subtotal - desconto + frete;

        if (total < 0) return 0;
        return total;
    }
}
```

OO - grafo de chamadas

Antes do CFG detalhado, um código OO pode ser lido como grafo de chamadas.

Esse grafo mostra colaboracoes, mas ainda não mostra todos os caminhos internos.

flowchart TD

T[Teste unitario] --> PS[PedidoService.fecharPedido]

PS --> SUB[Pedido.subtotal]

SUB --> IT[ItemPedido.total]

PS --> DESC[CalculadoraDesconto.calcular]

PS --> FRETE[CalculadoraFrete.calcular]

PS --> RET[retorno ou excecao]

OO - CFG do método de serviço

Agora descemos para o fluxo de controle do método fecharPedido.

flowchart TD

```
A([Inicio]) --> B{pedido vazio?}
B -- true --> C[throw IllegalArgumentException]
B -- false --> D[calcular subtotal]
D --> E[calcular desconto]
E --> F[calcular frete]
F --> G[total subtotal desconto frete]
G --> H{total menor que zero?}
H -- true --> I[return zero]
H -- false --> J[return total]
C --> K([Fim])
I --> K
J --> K
```

Solucao - McCabe do servico

Decisões em fecharPedido:

D1: pedido vazio?

D2: total < 0?

$V(G) = \text{decisões} + 1$

$V(G) = 2 + 1 = 3$

Caminhos independentes:

P1: pedido vazio -> excecao

P2: pedido não vazio -> total < 0 -> return 0

P3: pedido não vazio -> total >= 0 -> return total

OO - CFG da CalculadoraDesconto

A classe de desconto tem seu próprio grafo interno.

flowchart TD

```
A([Inicio]) --> B{cliente vip?}
B -- true --> C[return 10 por cento]
B -- false --> D{subtotal maior ou igual 500?}
D -- true --> E[return 5 por cento]
D -- false --> F[return zero]
C --> G([Fim])
E --> G
F --> G
```

Solucao - McCabe do desconto

Decisões em CalculadoraDesconto.calcular:

D1: cliente VIP?

D2: subtotal $\geq$ 500?

$V(G) = 3$

Caminhos:

P1: VIP $\rightarrow$ 10%

P2: não VIP e subtotal $\geq$ 500 $\rightarrow$ 5%

P3: não VIP e subtotal $<$ 500 $\rightarrow$ 0

OO - CFG da CalculadoraFrete

O switch da UF também gera alternativas de fluxo.

flowchart TD

A([Inicio]) --> B{switch UF}

B -- PR --> C[return 20]

B -- SP --> D[return 35]

B -- default --> E[return 50]

C --> F([Fim])

D --> F

E --> F

Solucao - McCabe do frete

Para um switch com 3 saídas relevantes:

UF = PR -> frete 20

UF = SP -> frete 35

UF = outra -> frete 50

$V(G) = 3$

Na prática: pelo menos um teste por alternativa relevante do switch.

OO - conclusao importante

No OO, o grafo de classes não substitui o CFG.

Ele ajuda a localizar onde os CFGs estão.

Fluxo recomendado:

1. olhar as classes e dependencias;
2. identificar o método de entrada;
3. montar o grafo de chamadas;
4. abrir o CFG dos métodos com decisões;
5. criar testes que cubram caminhos locais e colaboracoes.

OO - casos de teste sugeridos

CT01: pedido sem itens -> execucao

CT02: cliente comum, subtotal < 500, UF PR -> sem desconto, frete PR

CT03: cliente VIP, UF SP -> desconto VIP, frete SP

CT04: cliente comum, subtotal >= 500, UF RJ -> desconto por valor, frete default

CT05: forcar total < 0 com stub/mock de desconto -> return 0

OO - ponto didatico sobre testabilidade

O caminho total < 0 pode ser difícil de alcançar com as regras reais.

Isso mostra algo importante: algumas arestas do CFG podem exigir:

- dados especiais;
- objetos em estados específicos;
- stubs ou mocks;
- revisão da regra de negócio;
- remoção de código defensivo inalcançável.

Teste estrutural também revela problemas de projeto e testabilidade.

OO - exercicio em sala

Dado o código OO apresentado:

1. desenhe o grafo de relacionamento entre classes;
2. desenhe o grafo de chamadas a partir de fecharPedido;
3. desenhe o CFG de CalculadoraDesconto;
4. desenhe o CFG de CalculadoraFrete;
5. calcule $V(G)$ de cada método;
6. proponha testes unitarios e integrados;
7. indique quais caminhos exigem mock/stub.

Gabarito OO - resumo

PedidoService.fecharPedido:

$V(G) = 3$

CalculadoraDesconto.calcular:

$V(G) = 3$

CalculadoraFrete.calcular:

$V(G) = 3$

Pedido.subtotal:

possui loop sobre itens; $V(G) = 2$

Conclusao: a cobertura estrutural deve ser pensada por método e depois integrada pelo grafo de chamadas.

Regra mental

Ao ler um código, procure primeiro por estruturas que mudam o fluxo:

if, else, switch, for, while, do/while, return, break, continue, throw

Blocos básicos

Nem sempre precisamos transformar cada linha em um no.

Podemos agrupar instruções sequenciais em blocos básicos.

Um bloco básico é uma sequência sem desvio interno.

Exemplo simples - condição

Analise o código:

```
String classificar(int numero) {  
    if (numero >= 0) {  
        return "positivo";  
    }  
    return "negativo";  
}
```

Solucao do exemplo simples

Decisão: número ≥ 0

Caminhos:

P1: condição verdadeira -> positivo

P2: condição falsa -> negativo

Casos:

CT1: número = 10

CT2: número = -10

McCabe: $1 + 1 = 2$

Estrutural x funcional

No teste funcional, talvez valores-limite sejam importantes: -1, 0, 1.

No teste estrutural inicial, a preocupação é: a condição assumiu true e false?

As duas visões se complementam.

Exemplo - duas decisões aninhadas

Análise:

```
String analisar(int x) {  
    if (x > 0) {  
        if (x % 2 == 0) {  
            return "positivo par";  
        }  
        return "positivo impar";  
    }  
    return "nao positivo";  
}
```

Solucao - duas decisões aninhadas

Decisões:

D1: $x > 0$

D2: $x \% 2 == 0$

McCabe: $2 + 1 = 3$

Caminhos:

P1: D1 true, D2 true

P2: D1 true, D2 false

P3: D1 false

Casos: $x=2$, $x=3$, $x=-1$

Complexidade Ciclomática de McCabe

M McCabe forneceu uma medida da complexidade lógica do fluxo de controle.

Ela indica o tamanho de uma base de caminhos linearmente independentes.

Formula principal

Para um grafo conectado:

$$V(G) = E - N + 2$$

Onde:

E = número de arestas

N = número de nos

Formula prática

Em códigos estruturados simples, podemos conferir com:

$V(G) = \text{número de decisões} + 1$

Essa forma ajuda muito na aula, mas a formula do grafo continua sendo importante.

Caminho básico não e caminho mínimo

Use a expressão teste de caminho básico ou base mínima de caminhos independentes.

Não confundir com caminho mínimo de algoritmos de grafos.

Aqui, mínimo significa conjunto base suficiente para introduzir as arestas/caminhos independentes relevantes.

Caminhos independentes

Um caminho independente introduz pelo menos uma aresta ainda não usada pelos caminhos anteriores.

A ideia não é testar todas as combinações possíveis, mas montar uma base estrutural.

Exemplo guiado completo

Considere:

```
int desconto(int idade, boolean estudante) {  
    int desconto = 0;  
  
    if (idade >= 60) {  
        desconto = 20;  
    } else if (estudante) {  
        desconto = 10;  
    }  
  
    return desconto;  
}
```

Exemplo guiado - decisões e McCabe

Decisões:

D1: idade ≥ 60

D2: estudante

$$V(G) = 2 + 1 = 3$$

Precisamos de 3 caminhos independentes.

Exemplo guiado - caminhos e casos

P1: D1 true -> desconto 20

P2: D1 false, D2 true -> desconto 10

P3: D1 false, D2 false -> desconto 0

Casos:

CT1: idade=65, estudante=false, resultado=20

CT2: idade=20, estudante=true, resultado=10

CT3: idade=30, estudante=false, resultado=0

Exemplo - decisões independentes

Análise:

```
int calcular(int x, int y) {  
    int resultado = 0;  
  
    if (x > 0) {  
        resultado += 10;  
    }  
  
    if (y > 0) {  
        resultado += 20;  
    }  
  
    return resultado;  
}
```

Solucao - decisões independentes

Decisões:

D1: $x > 0$

D2: $y > 0$

McCabe: 3

Combinacoes possíveis: 4

VV, VF, FV, FF

A base de caminhos pode ter 3 caminhos. O quarto pode ser util funcionalmente, mas não e obrigatorio para a base.

Exemplo - loop

Análise:

```
int somar(int n) {  
    int soma = 0;  
  
    for (int i = 0; i < n; i++) {  
        soma += i;  
    }  
  
    return soma;  
}
```

Solucao - loop

Decisão: $i < n$

McCabe: $1 + 1 = 2$

Caminhos fundamentais:

P1: não entrar no laço

P2: entrar no laço

Casos: $n=0$ e $n=1$

Discussão prática: $n=2$ também pode ser útil para testar múltiplas iterações.

Exercicio 1 - aquecimento

Faca:

1. identifique decisões;
2. desenhe o grafo;
3. calcule $V(G)$;
4. identifique caminhos;
5. crie casos de teste.

```
String temperatura(double temp) {  
    if (temp > 30) {  
        return "quente";  
    }  
    return "normal";  
}
```

Exercicio 2

Determine CFG, N, E, V(G), caminhos independentes e entradas.

```
String classificar(int valor) {  
    if (valor < 0) {  
        return "negativo";  
    } else if (valor == 0) {  
        return "zero";  
    } else {  
        return "positivo";  
    }  
}
```

Exercicio 3

Pergunta adicional: existem combinacoes possíveis que não fazem parte da base?

```
double calcularFrete(double valor, boolean vip) {  
    double frete = 20;  
  
    if (valor >= 200) {  
        frete = 0;  
    }  
  
    if (vip) {  
        frete = frete * 0.5;  
    }  
  
    return frete;  
}
```

Exercicio 4

Determine N, E, V(G) e caminhos básicos.

```
int calcular(int x) {  
    int resultado = 0;  
  
    if (x > 10) {  
        if (x > 20) {  
            resultado = 2;  
        } else {  
            resultado = 1;  
        }  
    } else {  
        resultado = 0;  
    }  
  
    return resultado;  
}
```

Exercicio 5 - com repeticao

Identifique decisões, construa o grafo, calcule McCabe e escolha valores de n.

```
int contador(int n) {  
    int total = 0;  
  
    for (int i = 0; i < n; i++) {  
        if (i % 2 == 0) {  
            total++;  
        }  
    }  
  
    return total;  
}
```

Exercicio 6 - desafio

Quantas decisões? Qual $V(G)$? Quais caminhos independentes? Quais entradas?

```
double calcularValor(double valor, int quantidade, boolean clienteVip) {  
    double total = valor * quantidade;  
  
    if (quantidade >= 10) {  
        total = total * 0.9;  
    }  
  
    if (clienteVip) {  
        total = total * 0.95;  
    }  
  
    if (total > 1000) {  
        total = total - 50;  
    }  
  
    return total;  
}
```

Desafio final

Responda em 60 segundos:

- número de decisões
- $V(G)$
- número de caminhos independentes

```
String avaliar(int nota, int faltas) {  
    if (faltas > 20) {  
        return "reprovado";  
    }  
  
    if (nota >= 70) {  
        return "aprovado";  
    } else if (nota >= 40) {  
        return "exame";  
    } else {  
        return "reprovado";  
    }  
}
```

Gabarito - Exercício 1

Decisões: $\text{temp} > 30$

$V(G)=2$

Caminhos:

P1: $\text{temp} > 30$

P2: $\text{temp} \leq 30$

Casos: $\text{temp}=35$ e $\text{temp}=25$

Gabarito - Exercício 2

Decisões:

D1: $\text{valor} < 0$

D2: $\text{valor} == 0$

$V(G)=3$

Caminhos:

P1: $\text{valor} < 0$

P2: $\text{valor} \geq 0$ e $\text{valor} == 0$

P3: $\text{valor} \geq 0$ e $\text{valor} \neq 0$

Entradas: -1, 0, 1

Gabarito - Exercício 3

D1: valor ≥ 200

D2: vip

$V(G)=3$

Base possível:

P1: D1 false, D2 false

P2: D1 true, D2 false

P3: D1 false, D2 true

Caminho adicional possível: D1 true, D2 true

Gabarito - Exercício 4

D1: $x > 10$

D2: $x > 20$

$V(G)=3$

Caminhos:

P1: $x \leq 10$

P2: $x > 10$ e $x \leq 20$

P3: $x > 20$

Casos: $x=5$, $x=15$, $x=25$

Gabarito - Exercício 5

D1: $i < n$

D2: $i \% 2 == 0$

$V(G)=3$

Base:

P1: não entrar no for

P2: entrar e if verdadeiro

P3: entrar e if falso

Valores interessantes: $n=0$, $n=1$, $n=2$

Gabarito - Exercício 6

D1: quantidade ≥ 10

D2: clienteVip

D3: total > 1000

$V(G)=4$

Não é necessário testar todas as 8 combinações para a base estrutural.

Mas combinações adicionais podem ser relevantes por razões funcionais.

Gabarito - Desafio final

D1: faltas > 20

D2: nota >= 70

D3: nota >= 40

$V(G)=4$

Caminhos:

P1: faltas > 20

P2: faltas <= 20 e nota >= 70

P3: faltas <= 20 e nota < 70 e nota >= 40

P4: faltas <= 20 e nota < 40

Fechamento

Habilidade central:

Código -> Grafo -> Complexidade -> Caminhos -> Entradas -> Testes

Quando o aluno imagina o grafo antes de desenhá-lo, ele começou a analisar estruturalmente o software.

Roteiro sugerido de aula

- 15 min: caixa-preta x caixa-branca
- 20 min: código simples para grafo
- 25 min: representacoes explicitas das estruturas
- 15 min: McCabe
- 25 min: exemplos guiados
- 40 min: exercicios em duplas
- 20 min: correcao coletiva

Referencias

- Pressman, R. S. Engenharia de Software: Uma Abordagem Profissional. 2011.
- McCabe, T. J. A Complexity Measure. IEEE Transactions on Software Engineering, 1976.